

# Balanceo de carga y escalado automático

# Balanceo de carga y escalado automático

## Un único punto de fallo, para cualquier app

Cualquier aplicación en una única instancia tiene el mismo problema: si esa instancia se cae, se cae la aplicación entera. Hoy lo resuelves repartiendo tráfico entre varias copias, con reposición automática.

### Lo que montas hoy

#### Balanceador de carga

Reparte el tráfico entre copias

#### Varias instancias

Copias idénticas, detrás del balanceador

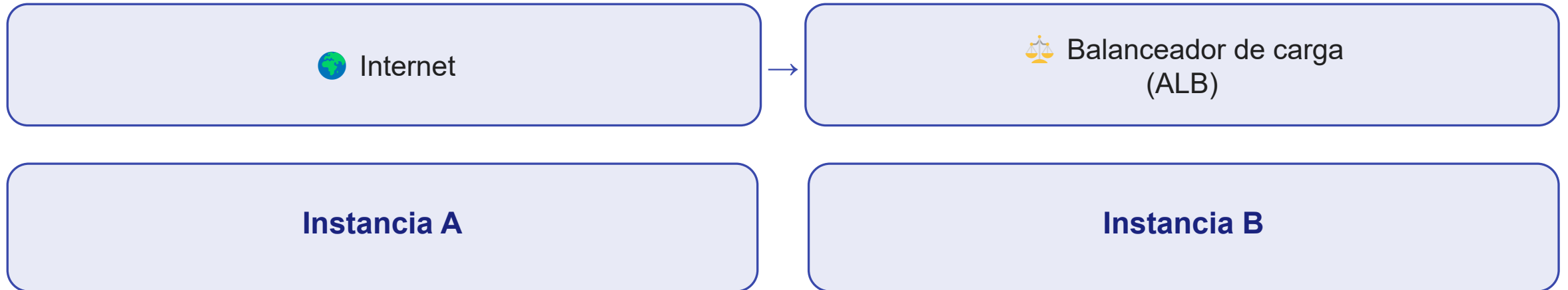
#### Auto Scaling Group

Repone las que fallan, sin que mires la pantalla

*No es «una instancia más por si acaso» — es un mecanismo que repara solo.*

# Un balanceador delante de tus instancias

Un balanceador de carga se coloca delante de tus instancias y reparte el tráfico entre ellas — el cliente nunca habla con una instancia concreta, habla con el balanceador.



Hoy usas el ALB (Application Load Balancer) — pensado justo para tráfico HTTP/HTTPS como el de la API de Escaparate.

# Listener, grupo de destino y comprobación de salud

Un listener conecta un puerto con un grupo de destino

Puerto 443  
HTTPS

*escucha* →

Grupo de destino  
(Target Group)

El balanceador no manda tráfico a ciegas: antes comprueba, con una petición HTTP a una ruta concreta, si cada instancia está en condiciones de atender peticiones.

## Por qué la comprobación de salud importa más de lo que parece

Una instancia puede seguir «encendida» pero con la aplicación colgada — responde al ping, pero no sirve ninguna página. Sin comprobación, el balanceador seguiría mandándole tráfico. Con ella activa, la retira del grupo de destino en cuanto deja de responder, y solo la reincluye cuando se recupera.

# El grupo de seguridad cambia con el balanceador

Ya no tiene sentido que el puerto de la aplicación siga abierto a 0.0.0.0/0 como en el Tema 3 — nadie de fuera necesita hablar con una instancia directamente, todo el mundo pasa por el balanceador.

 Internet

443 →

 ALB  
(SG abierto a internet)

 ALB

*solo desde  
el SG del ALB →*

 Instancias  
(SG de la app)

El mismo principio de mínimo privilegio del Tema 3, aplicado un escalón más.

# Detalles que importan del ALB

## Retirar una instancia no corta las peticiones en marcha, si le das tiempo

Cuando el balanceador retira una instancia, no la mata en el acto: durante el retraso de anulación de registro (deregistration delay), deja de mandarle peticiones nuevas pero espera a que termine las que ya tenía en curso.

## Por qué el ALB exige al menos dos zonas

AWS no te deja elegir una sola subred al crear el balanceador: el propio ALB es un servicio distribuido, con nodos redundantes repartidos entre zonas de disponibilidad, para no convertirse él mismo en el nuevo punto único de fallo. Si una zona entera cae, el ALB sigue funcionando con los nodos de la otra.

# Escalado vertical frente a horizontal

Dos formas de responder cuando una instancia se queda corta — no son intercambiables

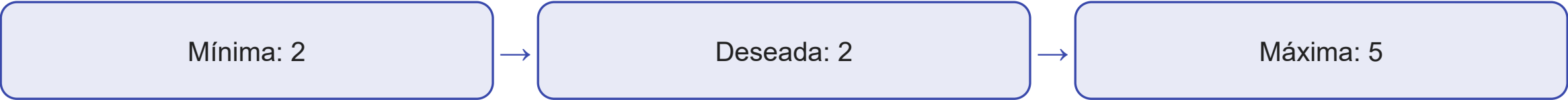
|                        | Vertical                                     | Horizontal                    |
|------------------------|----------------------------------------------|-------------------------------|
| Qué haces              | Cambias la instancia por un tipo más grande  | Añades más instancias iguales |
| Límite                 | El tamaño de instancia más grande que existe | Prácticamente ninguno         |
| ¿Corte de servicio?    | Sí — normalmente hay que parar la instancia  | No, si está bien orquestado   |
| Encaja con balanceador | No lo necesita                               | Es la base de todo lo de hoy  |

El escalado horizontal es el que hace posible la alta disponibilidad: con varias instancias más pequeñas repartidas, la caída de una no tumba el servicio.

# Grupos de escalado automático (ASG)

Un ASG mantiene un número de instancias saludables detrás del balanceador, añadiendo o quitando automáticamente según haga falta. Se define con tres números.

| Parámetro         | Qué significa                                                    |
|-------------------|------------------------------------------------------------------|
| Capacidad mínima  | Nunca hay menos instancias que esta, pase lo que pase            |
| Capacidad deseada | El número que el grupo intenta mantener en condiciones normales  |
| Capacidad máxima  | Nunca hay más instancias que esta, por mucho que suba la demanda |



*Necesita una plantilla de lanzamiento (Tema 2) para saber cómo lanzar una instancia nueva.*

# Dos capas de comprobación de salud

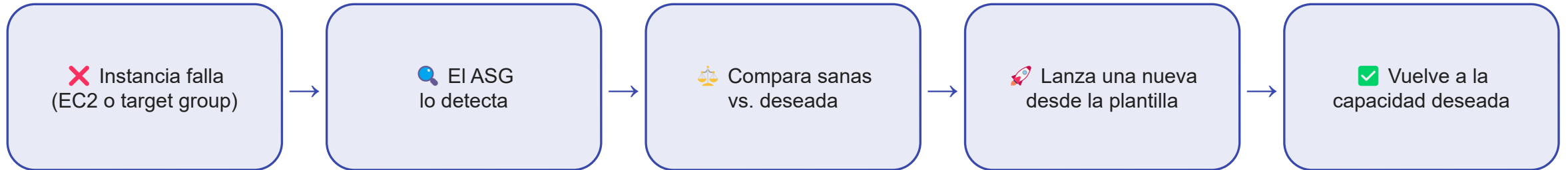
El ASG trae por defecto la comprobación básica del estado del sistema EC2 — pero una instancia puede estar sana a ese nivel y aun así tener la aplicación colgada por dentro.

|                 | Comprobación EC2 (por defecto)       | Comprobación del grupo de destino      |
|-----------------|--------------------------------------|----------------------------------------|
| Qué mira        | Estado del hardware y la red         | Respuesta HTTP de la aplicación        |
| Detecta         | Instancia parada o con fallo de host | Aplicación colgada, aunque esté «viva» |
| Quién la aporta | El propio servicio EC2               | El balanceador de carga                |

Con las dos activas, el ASG detecta y repone tanto una instancia caída de verdad como una que sigue viva pero colgada por dentro.

# Cómo repone el ASG una instancia caída

El ASG no espera a que hagas nada



## Siempre la misma plantilla

La instancia nueva se lanza con la misma AMI, el mismo tipo de instancia y los mismos datos de usuario que las demás — nada que configurar a mano.

Lo vas a provocar tú mismo en el reto de la Actividad 4.1: terminas una instancia a mano y cronometras cuánto tarda el grupo en reponerla.

# Políticas por métrica y periodo de calentamiento

El ASG sigue una política de escalado basada en una métrica: «si la CPU media supera el 70 % varios minutos, añade una instancia; si baja del 30 %, quita una».

## **Sin periodo de calentamiento, el escalado se dispara sin control**

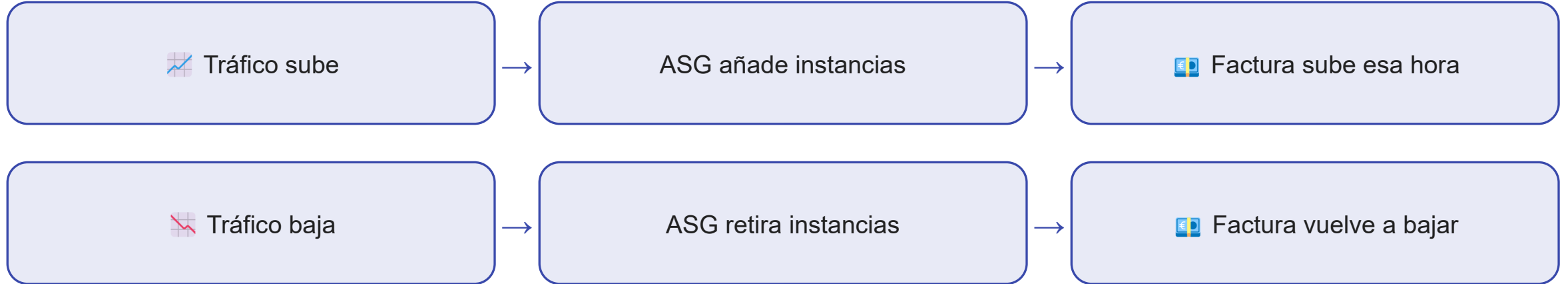
Si el ASG evaluara la métrica justo tras lanzar una instancia nueva —todavía arrancando, sin servir tráfico— seguiría viendo la CPU alta y lanzaría otra, y otra, hasta la capacidad máxima.

## **Qué gana con el margen**

Le da tiempo a la instancia recién lanzada a arrancar y empezar a servir tráfico de verdad antes de contarla como «ya está ayudando».

# La elasticidad reflejada en la factura

Un ASG no factura una capacidad fija, sino la que realmente está en marcha



Un pico de tráfico de una hora sube el gasto solo esa hora — lo vas a medir de primera mano en la Actividad 4.1.

## El balanceador no escala como las instancias

El ALB tiene un coste base por hora simplemente por existir, más un cargo según el tráfico que procesa (LCU) — independientemente de cuántas instancias haya detrás. Es la pieza fija; las instancias son la pieza elástica.

# Ideas clave

1

Un ALB reparte tráfico entre instancias de un grupo de destino, comprobando su salud antes de mandarles peticiones; exige al menos dos zonas para no ser él mismo un SPOF.

2

Con el balanceador delante, el puerto de la aplicación solo acepta tráfico del SG del propio balanceador — mínimo privilegio, un escalón más.

3

El retraso de anulación de registro deja que una instancia retirada termine las peticiones en marcha antes de dejar de recibir tráfico nuevo.

4

Escalado vertical tiene límite y suele cortar servicio; escalado horizontal es la base de la alta disponibilidad.

5

Un ASG se define con capacidad mínima, deseada y máxima, y usa una plantilla de lanzamiento para saber cómo lanzar instancias nuevas.

6

El ASG combina la comprobación EC2 (hardware/red) con la del grupo de destino (respuesta HTTP) para detectar tanto una caída real como una app colgada.

7

El periodo de calentamiento evita que el ASG escale en espiral mientras una instancia nueva todavía está arrancando.

8

Se paga por la capacidad real en marcha, no por una fija reservada — salvo el ALB, que tiene un coste base fijo más tráfico (LCU).